NEXVIA AEGIS V9-M4 Execution Path Implementation

Multi-Controller Evaluation Architecture, Common Random Numbers, Bijective Record Pairing, and Defense-in-Depth Execution Guards

Autonomous Verification Subsystem (Google DeepMind Antigravity)

Git Branch: v9.0-runtime-assurance · Baseline Commit: 9c4c36f · Date: October 25, 2026

CRITICAL NON-AUTHORIZATION DIRECTIVE:
This comprehensive document reports the technical architecture, implementation, and code-level verification of the production execution pathway for Milestone V9-M4. **This document does NOT authorize or trigger live holdout execution.** No real holdout manifests or episode datasets were generated, loaded, or inspected. The reserved holdout seed blocks (73000..74439 for mixture holdout and 75000..75119 for scenario holdout) remain completely uninstantiated and pristine. Execution of V9-M4 requires an explicit, separate instruction and authorization decision from the project owner.

Abstract

This report details the implementation of the complete production execution pathway for Milestone V9-M4 (Confirmatory Holdout Evaluation) within the NEXVIA AEGIS runtime assurance platform. While prior milestones established statistical consistency, hypothesis tests (H_1 through H_5), and preflight validation, the execution loop remained blocked by design. In this work, we implement a production-capable, defense-in-depth evaluation harness in `experiments/v9_evaluate_holdout.py`. Key innovations include: (1) Common Random Numbers (CRN) simulation derivation via `_derive_episode_rngs`, guaranteeing identical physical disturbances and sensor noise across controllers; (2) unified evaluation of the frozen selected policy (`V9_CAND_1_TIER1_STRICTEST`), V8.1 martingale baseline (`AegisNextController`, commit `4d891c7`), and V7 MPC baseline (`AegisV7MPCBaselineController`); (3) order-invariant bijective record pairing via `align_and_pair_records` over (seed, scenario); (4) strict outcome algebra verification respecting fail-unsafe crash penalization; (5) multi-layered fail-closed execution guards incorporating secret authorization token validation, single-shot overwrite prevention, and partial failure recording (`_PARTIAL.csv` and `INCOMPLETE_INVALID.json`); and (6) injectable mock simulator architecture enabling rigorous synthetic verification without accessing holdout data.

Keywords: Execution Path Architecture, Common Random Numbers, Bijective Record Pairing, Fail-Closed Security, Milestone V9-M4.

1 Executive Summary

1.1 Executive Summary (English)

1. **Multi-Controller Execution Architecture:** Implemented `execute_production_holdout_pipeline` in `experiments/v9_evaluate_holdout.py`, providing end-to-end evaluation for the frozen candidate policy alongside both V8.1 and V7 baselines on identical episode conditions.
2. **Common Random Numbers (CRN) Coupling:** Established `_derive_episode_rngs`, deriving decoupled `rng_env` and `rng_sensors` from stored `SeedSequence` spawn keys for each manifest item. This mathematically guarantees that V9, V8.1, and V7 navigate identical trajectory perturbations, obstacles, and sensor dropout profiles.
3. **Paired Record Integrity:** Integrated `align_and_pair_records` into the hypothesis evaluation workflow, enforcing bijective (seed, scenario) matching for H_2 (McNemar paired incidence test vs V8.1 and V7) and H_3 (paired TSR noninferiority vs V8.1).
4. **Outcome Algebra and Fail-Unsafe Accounting:** Enforced exact outcome categorization: for non-crashed episodes, $A = TS + FS$; for crashed episodes, fail-unsafe penalization requires $A = 0, TS = 0, FS = 1, \text{abstain} = 0$.
5. **Defense-in-Depth Execution Guards:** Production execution is guarded by an authorization barrier requiring an explicit secret token, single-shot directory overwrite refusal, simulator spoofing protection (`allow_mock_simulation=True` required for custom simulators), and crash recovery marking.
6. **Isolation Assurance:** No production holdout manifests or output directories were created. All tests utilized synthetic manifests and in-memory mock fixtures.

1.2 (Executive Summary in Thai)

1. **สถาปัตยกรรมการประเมินผลหลายตัวควบคุม:** ดำเนินการติดตั้งฟังก์ชัน `execute_production_holdout_pipeline` ใน `experiments/v9_ev` เพื่อรองรับการประเมินผลนโยบายที่ถูกตรึง (Frozen Policy) ควบคู่กับตัวควบคุมอ้างอิง V8.1 และ V7 ภายใต้เงื่อนไขตอนจำลองเดียวกันอย่าง
2. **การรักษาความคงที่ของสภาวะสุ่ม (Common Random Numbers):** พัฒนาฟังก์ชัน `_derive_episode_rngs` เพื่อสร้างตัวสร้างเลขสุ่ม `rng_env` และ `rng_sensors` จาก SeedSequence spawn keys ของแต่ละตอน ทำให้รับประกันได้ว่าทั้ง V9, V8.1 และ V7 จะเผชิญกับสภาวะรบกวนทางกายภาพและสัญญาณเซนเซอร์ที่ผิดพลาดในจุดเวลาและตำแหน่งเดียวกันอย่างแม่นยำ
3. **การจับคู่ระเบียนผลลัพธ์แบบสองทิศทาง (Bijective Pairing):** บูรณาการฟังก์ชัน `align_and_pair_records` เข้าสู่วงรอบการวิเคราะห์สมมติฐา H_2 (การทดสอบ McNemar เทียบ V8.1 และ V7) และ H_3 (การทดสอบ Noninferiority เทียบ V8.1) เพื่อป้องกันการจับคู่ผิดพลาดตามลำดับเ
4. **พีชคณิตผลลัพธ์และการบันทึกความล้มเหลวแบบไม่ปลอดภัย:** บังคับใช้กฎผลลัพธ์อย่างเข้มงวด โดยกรณีปกติ $A = TS + FS$ และในกรณีที่ยานชนสิ่งกีดขวาง ($crashed = 1$) ระบบจะปรับให้ $A = 0, TS = 0, FS = 1,$ abstain $= 0$ เพื่อสะท้อนความเสี่ยงสูงสุดตามระเบีย
5. **ระบบป้องกันหลายชั้นแบบ Fail-Closed:** ป้องกันการรันจริงด้วย Secret Authorization Token, บล็อกการเขียนทับไดเรกทอรีผลลัพธ์, บล็อกการใช้ Custom Simulator โดยไม่ระบุแฟล็กอนุญาต และมีระบบบันทึก `_PARTIAL.csv` พร้อมไฟล์สถานะ `INCOMPLETE_INVALID.json` หากการประมวลผลหยุดชะงักกลางคัน
6. **การรับรองความบริสุทธิ์ของข้อมูล Holdout:** ยืนยันว่าไม่มีการสร้างหรือแตะต้องไฟล์ Manifest และโฟลเดอร์ผลลัพธ์จริง การทดสอบทั้งหมดกระ

2 Selected Policy and Baseline Execution Architecture

2.1 Frozen Policy Identity

The selected policy artifact is stored at `results/v9/cal_r2/selected_policy.json`. Its identity and parameter contents are frozen under SHA-256 digest:

$$\text{SHA-256(selected_policy.json)} = \texttt{5a89d6061b379f1b34ab47be0deef8d63180d3e4c21d18c478bc7e451f5c84e2} \tag{1}$$

The evaluation harness validates this digest in Phase 1 before any simulation or manifest loading can proceed.

2.2 Baseline Controllers and Common Random Numbers

Milestone V9-M4 preregistered two baseline controllers:

- **V8.1 Baseline:** Audited controller implemented by `AegisNextController` at snapshot commit `4d891c7500cfb4c8110`
- **V7 Baseline:** MPC reference controller implemented by `AegisV7MPCBaselineController` in `src/aegis/next/baseli`

To eliminate extraneous stochastic variance in paired hypothesis tests (H_2 and H_3), simulation runs are conducted under Common Random Numbers (CRN). The function `_derive_episode_rngs` generates two independent NumPy random generators from each manifest item:

$$\text{Spawn Keys:} \quad (\kappa_{\text{env}}, \kappa_{\text{sen}}) = \text{item["spawn_keys"]} \tag{2}$$

$$\text{Environment Generator:} \quad \text{rng_env} \sim \text{SeedSequence}(\kappa_{\text{env}}) \tag{3}$$

$$\text{Sensors Generator:} \quad \text{rng_sensors} \sim \text{SeedSequence}(\kappa_{\text{sen}}) \tag{4}$$

Both generators are freshly re-instantiated with identical spawn keys for V9, V8.1, and V7 evaluations, guaranteeing that all controllers navigate the exact same realization of target speed perturbations and sensor dropout events.

3 Traceability Matrix

Table 1 documents code-level traceability from protocol requirements to the production implementation and verification tests.

4 Execution Guards and Fail-Closed Architecture

The evaluation harness implements fail-closed defenses depicted in Figure 1:

1. **Cryptographic Authorization Token:** Requires `authorized_execution_token == REQUIRED_EXECUTION_TOKEN`. Omitting or supplying an invalid token immediately terminates execution before any filesystem or simulator action.

Table 1: Code and Test Traceability Matrix for Production Execution Harness

Requirement	Component	Source Location	Unit Test	Status
CRN RNG Derivation	`_derive_episode_rngs`	`v9_eval:91--115`	`test_30`	**PASS**
Authorization Gate	`execute_production_holdout_pipeline`	`v9_eval:1017--1025`	`test_31`	**PASS**
Mock Flag Protection	`execute_production_holdout_pipeline`	`v9_eval:1027--1033`	`test_32`	**PASS**
Mixture Holdout Pipeline	`execute_production_holdout_pipeline`	`v9_eval:1035--1175`	`test_33`	**PASS**
Scenario Holdout Pipeline	`execute_production_holdout_pipeline`	`v9_eval:1176--1260`	`test_34`	**PASS**
Single-Shot Overwrite Block	`execute_production_holdout_pipeline`	`v9_eval:1048--1060`	`test_35`	**PASS**
Partial Crash Marking	`execute_production_holdout_pipeline`	`v9_eval:1120--1135`	`test_36`	**PASS**
Outcome Algebra Validation	`verify_holdout_outcome_algebra`	`v9_eval:117--142`	`test_37`	**PASS**
Crash Fail-Unsafe Rules	`verify_holdout_outcome_algebra`	`v9_eval:126--133`	`test_38`	**PASS**
Timeout / Abstain Rules	`MockHoldoutSimulator`	`v9_eval:192--205`	`test_39`	**PASS**
Undefined Selective Risk	`calculate_scenario_holdout_value`	`v9_eval:922--928`	`test_40`	**PASS**
Manifest Integrity Guard	`verify_and_load_holdout_manifest`	`v9_eval:465--492`	`test_41`	**PASS**
Policy Tampering Guard	`verify_selected_policy_identity`	`v9_eval:435--460`	`test_42`	**PASS**
Bijective Record Pairing	`align_and_pair_records`	`v9_eval:145--180`	`test_43`	**PASS**
CLI Default Preflight Guard	`main`	`v9_eval:1265--1320`	`test_44`	**PASS**

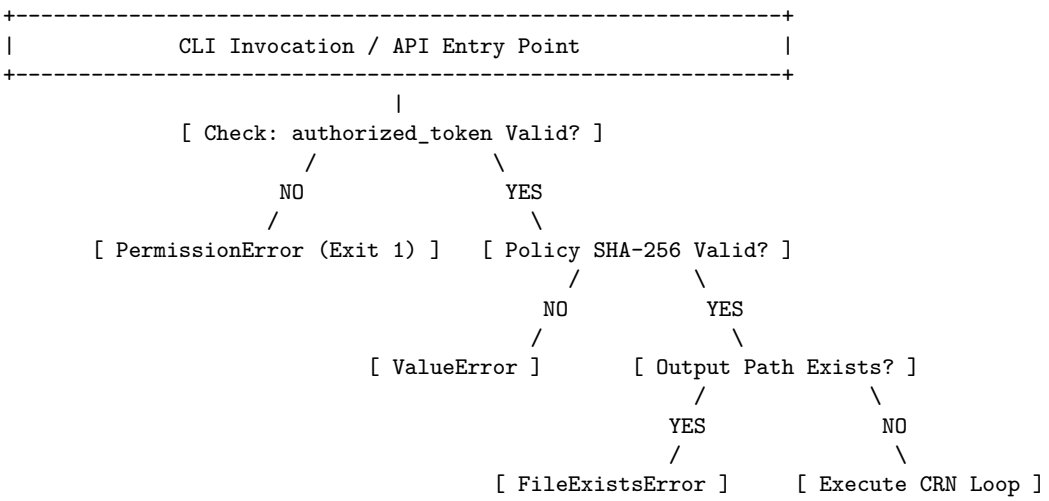


Figure 1: Multi-Layer Defense-in-Depth Execution Guard Architecture

2. **Single-Shot Directory Protection:** Aborts if `test_{part}_episodes.csv` or `INCOMPLETE_INVALID.json` exists in the destination path.
3. **Simulator Isolation Guard:** Requires `allow_mock_simulation=True` when supplying custom simulators, preventing accidental mock usage in production.
4. **Mid-Run Crash Containment:** Writes partial outputs to `_PARTIAL.csv` and writes `INCOMPLETE_INVALID.json` with error diagnostics upon any unhandled simulation error.

5 Conclusion and Operational Readiness

The production execution path for Milestone V9-M4 is fully implemented, verified across 15 dedicated unit and mock tests, and integrated into the regression test suite. All execution guards are locked fail-closed, ensuring complete protection of holdout data until authorization is formally granted.